

Desafio TechFlow Solutions

Objetivo

O foco deste projeto é ser uma atividade entregável da faculdade, o objetivo é fazer um sistema de Cadastro e Login usando Python.

TechFlow Solutions sendo o nome fictício de uma empresa, especializada em Soluções em Software.

Desafio Proposto

"Aplicando os conceitos de Engenharia de Software para planejar, criar, e gerenciar um repositório no GitHub que simule o desenvolvimento do sistema. A atividade envolverá desde a organização do repositório até a implementação de funcionalidades básicas, uso de ferramentas de controle de qualidade e gestão de mudanças.

A TechFlow Solutions, uma empresa fictícia especializada em soluções de software, foi contratada para desenvolver um sistema de gerenciamento de tarefas baseado em metodologias ágeis. O cliente, uma startup de logística, busca um sistema que permita acompanhar o fluxo de trabalho em tempo real, priorizar tarefas críticas e monitorar o desempenho da equipe"

Escopo do Projeto

Usando Python, com a biblioteca Flask, o usuário poderá realizar Login e Cadastro em um site da empresa, sendo capaz de editar informações e removê-las (deletar cadastro).

1. Cadastro e Login como tarefa principal
2. Deve ser capaz de redefinir a senha
3. Tela depois de Login para localizar o usuário.

O escopo do projeto tem como foco aplicar as metodologias e testes de funcionalidade com a maior prioridade, portanto não haverá diversas funções / adições. Ou seja, inicialmente só as funções básicas em Flask.

Para esse projeto, não será usado Banco de Dados, inicialmente guardará os resultados em memória (ou seja, temporários) para efeitos de demonstração.

Além disso, também será priorizado os testes do sistema de login/cadastro com a validação do GitHub Actions e outras validações.

A segurança não foi o foco principal do projeto, pois o objetivo é validar fluxo de autenticação, estrutura do sistema e integração com ferramentas de CI/CD.

Durante a produção do projeto, foi identificado a necessidade de validação forte de senha por motivos de segurança, ou seja, requisitando que os usuários cadastrem senhas fortes. Requisitos: Mínimo de 8 caracteres, pelo menos 1 número e 1 letra maiúscula.

Metodologia Aplicada

A metodologia aplicada será a "Metodologia Ágil", com atualizações incrementais e atualizações constantes, diferente de outros métodos como o Cascata.

A organização das tarefas foi feita utilizando Kanban no GitHub Projects.

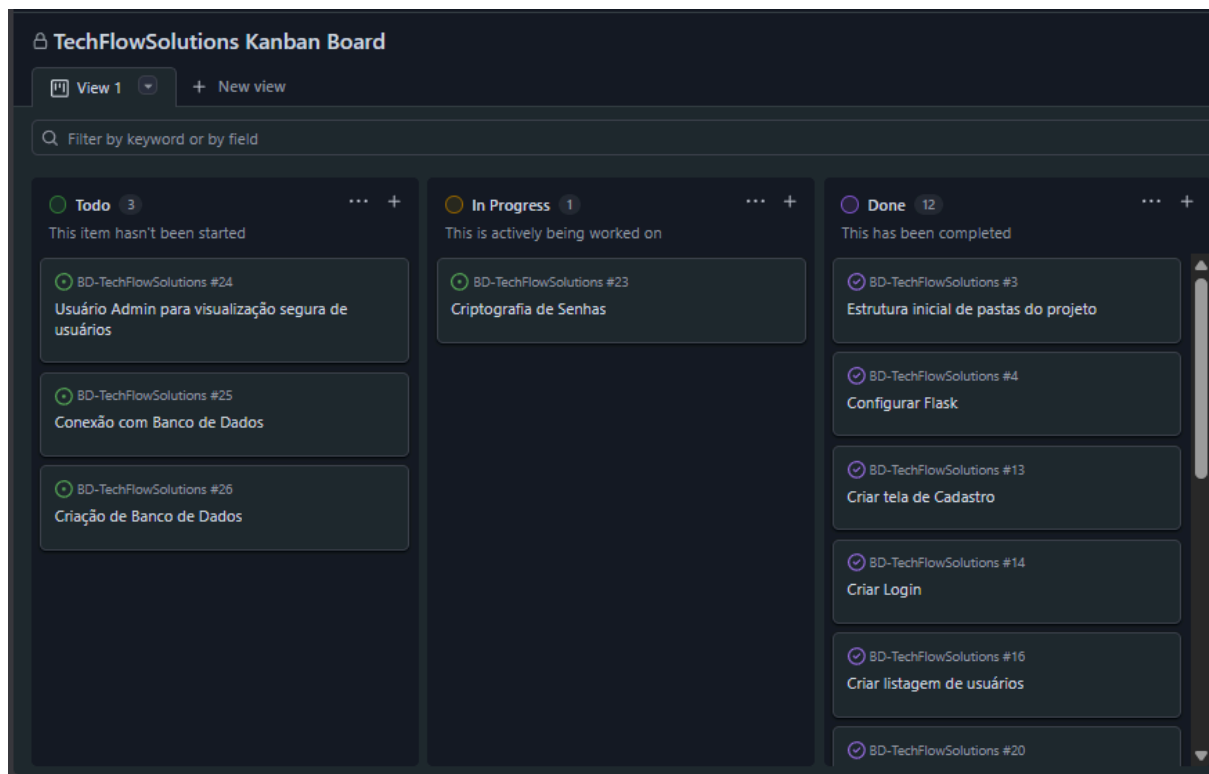
Tecnologias

- Python
- Flask
- PyTest
- GitHub Actions

Controle de Qualidade

O projeto utiliza GitHub Actions para automação de testes com PyTest, garantindo validação contínua do sistema.

Quadro Kanban



Todo:

Essa parte atualmente possui apenas 3 tarefas, que seriam os próximos passos conforme o projeto fosse sendo levado à frente, sendo levado em consideração prioridade.

In Progress:

Possui uma tarefa de maior prioridade: Criptografia de Senhas, já que atualmente, há um defeito enorme em que o usuário não tem senhas salvas de forma segura.

Done:

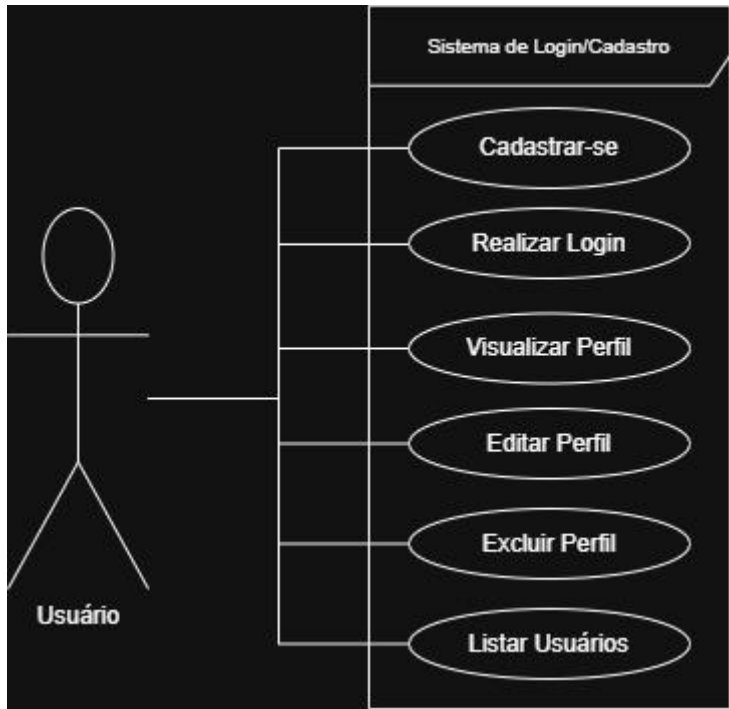
Todas as tarefas concluídas durante o projeto..

Importância da Modelagem na Engenharia de Software

Na engenharia de software, é importante organizar e modelar o sistema primeiro para garantir que, primeiramente o escopo e objetivos iniciais estejam bem estabelecidos, assim evitando problemas de comunicação ou má organização por parte da equipe. Ao modelar um sistema de login, por exemplo, fica fácil a compreensão de como funcionará o fluxo de cadastro e login e a interação com o sistema (back-end).

Além disso, ao documentar visualmente como um sistema funciona garante que os desenvolvedores mantenham o código compreensível, fácil de apresentar a novos integrantes (caso necessário), além de servir como estrutura ou base para novas adições conforme o projeto se torne mais complexo.

Linguagem de Modelagem Unificada (UML)



Casos de Uso

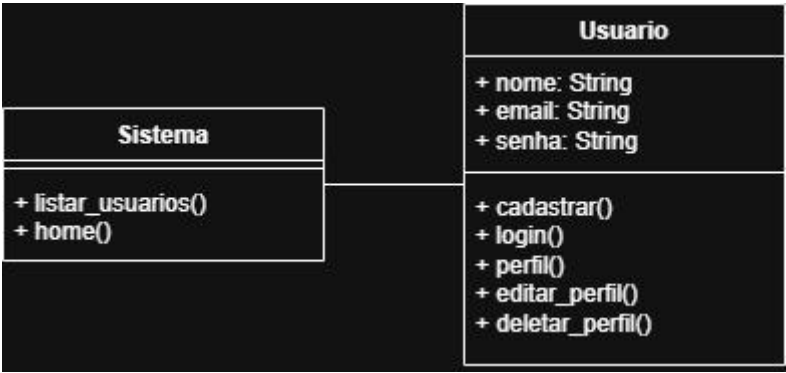
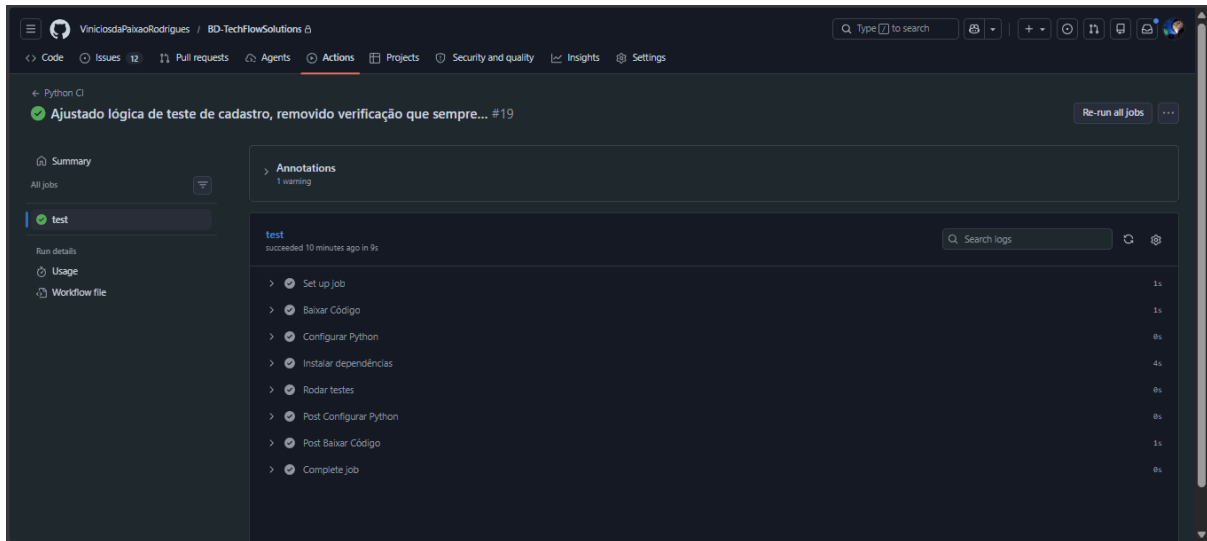


Diagrama de Classes

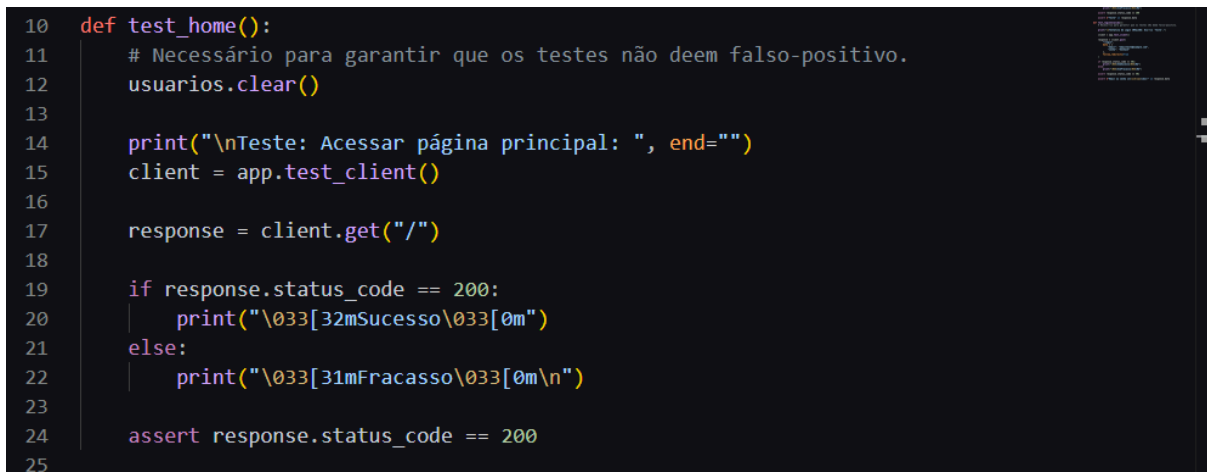
Testes (PyTest + GitHub Actions)



Durante o desenvolvimento deste projeto, foi utilizado a biblioteca PyTest, que localiza o arquivo que comece com “test” e seja “.py”, e depois executa todas as funções que começam com “test”, assim permitindo verificar e testar um fluxo de ações simuladas.

GitHub Actions em suma cria o ambiente e roda os testes com o PyTest.

Testes Utilizados



Este teste verifica se é possível acessar a página “home”, no caso a “index.html” do projeto. Mesmo sendo um teste simples, permite verificar algumas coisas:

1. Se o web server está rodando;
2. Se é possível encontrar a página principal (sem acessar a página com sucesso, mas sem nada em html feito, por exemplo).
3. Se há algum erro de conexão, rotas ou coisa do tipo atual no servidor.

```

26 def test_cadastro():
27     # Necessário para garantir que os testes não deem falso-positivo.
28     usuarios.clear()
29
30     print("\nTentativa de Cadastro: Usuário 'Teste'.\nResultado: ", end="")
31
32     client = app.test_client()
33
34     response = client.post(
35         "/cadastro",
36         data={
37             "nome": "Teste",
38             "email": "emailteste@example.com",
39             "senha": "Senha123"
40         },
41         follow_redirects=True
42     )
43
44     if response.status_code == 200:
45         print("\033[32mSucesso\033[0m")
46     else:
47         print("\033[31mFracasso\n\033[0m")
48
49     assert response.status_code == 200
50
51     assert any(
52         u["email"] == "emailteste@example.com"
53         for u in usuarios
54     )
55

```

Já este, verifica se é possível cadastrar um usuário teste com as seguintes informações:

Nome do usuário: Teste

Email: emailteste@example.com

Senha: Senha123

Por que é importante? Caso tenha algum problema no fluxo de cadastro, será possível verificar de imediato o que está ocorrendo, além de apurar o troubleshooting.

```

56 def test_login():
57
58     print("\nTentativa de Login: Usuário 'Teste'.")
59
60     client = app.test_client()
61
62     response = client.post(
63         "/login",
64         data = {
65             "email": "emailteste@example.com",
66             "senha": "Senha123"
67         },
68         follow_redirects=True
69     )
70
71     if response.status_code == 200:
72         print("\033[32mSucesso\033[0m")
73     else:
74         print("\033[31mFracasso\033[0m")
75
76     assert response.status_code == 200
77
78     assert b"Teste" in response.data
79

```

Similar ao teste de Cadastro, verifica se é possível logar com o mesmo usuário criado.

```

80 def test_loginInvalido():
81     # Necessário para garantir que os testes não deem falso-positivo.
82
83     print("\nTentativa de Login INVÁLIDA: Usuário 'Teste'.")
84
85     client = app.test_client()
86
87     response = client.post(
88         "/login",
89         data = {
90             "email": "emailteste@example.com",
91             "senha": "Senha13"
92         },
93         follow_redirects=True
94     )
95
96     if response.status_code == 401:
97         print("\033[32mSucesso\033[0m")
98     else:
99         print("\033[31mFracasso\033[0m")
100
101     assert response.status_code == 401
102
103     assert b"Email ou senha inv\x03\x0a\x0didos!" in response.data

```

Este verifica se é possível logar com um usuário inválido, importante caso tenha algum problema e seja possível logar com um usuário errado, ou a janela tenha algum problema.

Commits Relevantes

main

Commits on May 28, 2026

Ajustado lógica de teste de cadastro, removido verificação que sempre retornava em erro (Esperava resultado impossível)

ViniciosdaPaixaoRodrigues committed 21 minutes ago · 1 / 1

706afa0

Commit mais recente (do momento que este documento foi escrito), onde foi corrigido o workflow e ajustado a lógica do teste de cadastro. Este commit também explica de forma resumida o que foi corrigido.

Commits on May 23, 2026

task: Reorganizado README para facilitar leitura, removido inconsistências e atualizadas diversas partes. Também foi atualizado os htmls do Template para ficarem consistentes.

ViniciosdaPaixaoRodrigues committed 5 days ago · 1 / 1

0a111cf

Neste commit, foi feito a reorganização do README, essencial para que novos clientes em potenciais possam testar o software enquanto compreendem o projeto e entendem a visão/decisões/motivos do que algumas coisas foram feitas.

task: Adicionado exigências de senha forte. min 8 caracteres, 1 maiúscula e número

ViniciosdaPaixaoRodrigues committed 5 days ago · 1 / 1

7fa4a8e

Este commit comenta sobre a inclusão da exigência de senha forte aplicada, conforme mudança de escopo mencionada neste documento.

task: Corrigido workflow, reajustado requirements para somente o necessário

ViniciosdaPaixaoRodrigues committed 5 days ago · 1 / 1

cd84f71

task: movido requirements.txt para a raiz do projeto, permitindo o GitHub Actions rodar normalmente.

ViniciosdaPaixaoRodrigues committed 5 days ago · 0 / 1

1c6e801

task: Criado workflow GitHub Actions

ViniciosdaPaixaoRodrigues committed 5 days ago · 0 / 1

ac9db09

Sequência que demonstra quando o Workflow foi originalmente criado, além de mostrar o processo até que o Workflow estivesse funcional, ao menos naquele momento.



Um dos commits iniciais, relevante no sentido de entender onde o projeto começou a ser desenvolvido, dando uma visão de o que/quando estava sendo feito.